Laboratorio 8 - Máquinas Vectoriales de Soporte (SVM)

Luis Gonzalez 23353 - Joel Jaquez 23369 - Fabian Morales 23267

2026-04-24

Actividad 1 - Conjuntos de entrenamiento y prueba

```r
library(dplyr)
library(readr)
library(tidyr)
library(ggplot2)
library(caret)
library(e1071)
library(reshape2)
```

```r
load("datos/listings.RData")
```

```r
# Mismos conjuntos de entrenamiento y prueba de todos los labs anteriores
# set.seed(123), split 80/20, mismas 6 variables y filtros
listings_clean_comp <- listings %>%
  mutate(price = parse_number(price)) %>%
  select(-c(listing_url, scrape_id, picture_url, host_url,
            host_thumbnail_url, host_picture_url, description,
            neighborhood_overview, host_about, amenities))

set.seed(123)
total_filas  <- nrow(listings_clean_comp)
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)
train_comp   <- listings_clean_comp[ indice_train, ]
test_comp    <- listings_clean_comp[-indice_train, ]

train_modelo <- train_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

test_modelo <- test_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
```

```r
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

cat("Filas entrenamiento:", nrow(train_modelo), "\n")
```

```
## Filas entrenamiento: 48442
```

```r
cat("Filas prueba:       ", nrow(test_modelo),  "\n")
```

```
## Filas prueba:        12173
```

Para garantizar una comparación válida con todos los modelos desarrollados a lo largo del curso, se reutilizaron exactamente los mismos conjuntos de entrenamiento y prueba generados desde el Laboratorio 3. Utilizando set.seed(123) y una división 80/20, el conjunto de entrenamiento quedó con 48,442 observaciones y el de prueba con 12,173, ambos con las mismas 6 variables: price, accommodates, bedrooms, bathrooms, number_of_reviews y review_scores_rating, aplicando los mismos filtros de precio menor a $1,000 y habitaciones menores o iguales a 10.

Actividad 2 - Exploración de datos y transformaciones para SVM

```r
# Resumen estadístico de las variables
summary(train_modelo)
```

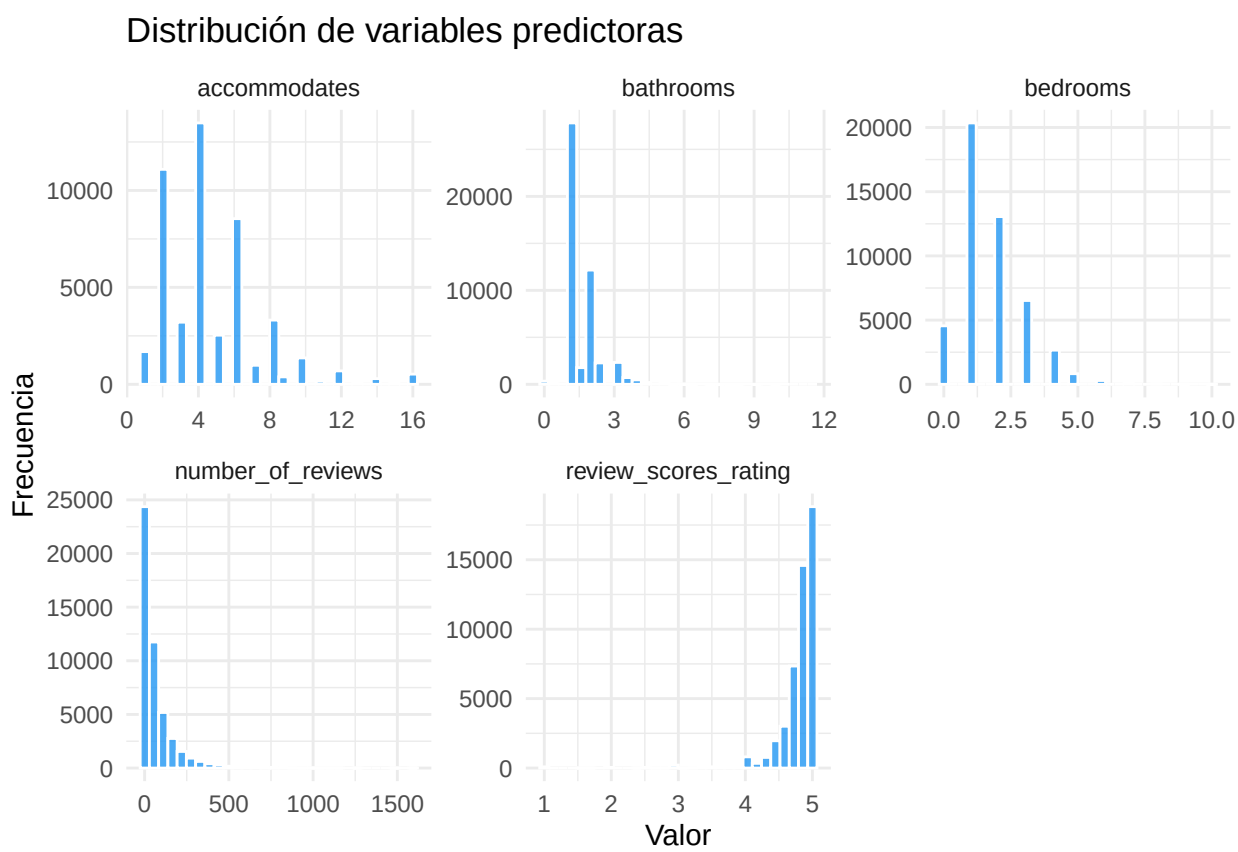
```
##      price        accommodates       bedrooms        bathrooms
##  Min.   :  8.0   Min.   : 1.000   Min.   : 0.000   Min.   : 0.000
##  1st Qu.:114.0   1st Qu.: 2.000   1st Qu.: 1.000   1st Qu.: 1.000
##  Median :178.0   Median : 4.000   Median : 1.000   Median : 1.000
##  Mean   :228.6   Mean   : 4.686   Mean   : 1.731   Mean   : 1.535
##  3rd Qu.:285.0   3rd Qu.: 6.000   3rd Qu.: 2.000   3rd Qu.: 2.000
##  Max.   :999.0   Max.   :16.000   Max.   :10.000   Max.   :11.500
##  number_of_reviews review_scores_rating
##  Min.   :   1.00   Min.   :1.000
##  1st Qu.:   7.00   1st Qu.:4.750
##  Median :  28.00   Median :4.890
##  Mean   :  67.96   Mean   :4.802
##  3rd Qu.:  85.00   3rd Qu.:5.000
##  Max.   :1592.00   Max.   :5.000
```

```r
# Correlación entre predictores
cor_matrix <- cor(train_modelo)
print(round(cor_matrix, 3))
```

```
##                      price accommodates bedrooms bathrooms number_of_reviews
## price                1.000        0.548    0.525     0.536            -0.139
## accommodates         0.548        1.000    0.831     0.672            -0.083
## bedrooms             0.525        0.831    1.000     0.713            -0.102
## bathrooms            0.536        0.672    0.713     1.000            -0.130
```

```
## number_of_reviews    -0.139       -0.083   -0.102    -0.130              1.000
## review_scores_rating  0.091        0.054    0.070     0.042              0.086
##                      review_scores_rating
## price                               0.091
## accommodates                        0.054
## bedrooms                            0.070
## bathrooms                           0.042
## number_of_reviews                   0.086
## review_scores_rating                1.000
```

```
# Distribución de las variables predictoras
train_modelo %>%
  select(-price) %>%
  tidyr::pivot_longer(everything(),
                      names_to  = "Variable",
                      values_to = "Valor") %>%
  ggplot(aes(x = Valor)) +
  geom_histogram(bins = 30, fill = "#2196F3",
                 color = "white", alpha = 0.8) +
  facet_wrap(~ Variable, scales = "free") +
  labs(title = "Distribución de variables predictoras",
       x = "Valor", y = "Frecuencia") +
  theme_minimal()
```



```r
# SVM requiere normalización obligatoria
# Verificar escalas de las variables
cat("=== Rangos de variables ===\n")
```

```
## === Rangos de variables ===
```

```r
cat("accommodates      :", min(train_modelo$accommodates),
    "-", max(train_modelo$accommodates), "\n")
```

```
## accommodates      : 1 - 16
```

```r
cat("bedrooms          :", min(train_modelo$bedrooms),
    "-", max(train_modelo$bedrooms), "\n")
```

```
## bedrooms          : 0 - 10
```

```r
cat("bathrooms         :", min(train_modelo$bathrooms),
    "-", max(train_modelo$bathrooms), "\n")
```

```
## bathrooms         : 0 - 11.5
```

```r
cat("number_of_reviews :", min(train_modelo$number_of_reviews),
    "-", max(train_modelo$number_of_reviews), "\n")
```

```
## number_of_reviews : 1 - 1592
```

```r
cat("review_scores_rating:", min(train_modelo$review_scores_rating),
    "-", max(train_modelo$review_scores_rating), "\n")
```

```
## review_scores_rating: 1 - 5
```

La exploración de los datos revela características importantes que justifican las transformaciones necesarias para SVM. El resumen estadístico muestra que las variables tienen escalas completamente distintas: number_of_reviews va de 1 a 1,592 mientras que review_scores_rating va de 1 a 5, y accommodates va de 1 a 16. Esta diferencia de escalas es crítica para SVM ya que el algoritmo calcula distancias entre puntos en el espacio de características, por lo que variables con rangos mayores dominarían completamente el cálculo. La normalización es obligatoria y se aplicará con preProcess = c("center", "scale") igual que en KNN.

Las distribuciones confirman este problema: number_of_reviews está fuertemente sesgada a la derecha con la mayoría de propiedades concentradas cerca de 0 y algunos valores extremos hasta 1,592, mientras que review_scores_rating se concentra entre 4.5 y 5. Las variables de capacidad (accommodates, bedrooms, bathrooms) tienen distribuciones discretas concentradas en valores bajos.

La matriz de correlación muestra multicolinealidad moderada entre accommodates, bedrooms y bathrooms (correlaciones de 0.67 a 0.83), similar a lo observado en labs anteriores. Esto no impide usar SVM pero sí refuerza la necesidad de normalización para que ninguna variable domine el hiperplano de separación.

Actividad 3 - Variable categórica de precio

```r
# Mismos cortes P33=136 y P67=250 de todos los labs anteriores
precios_validos <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)

p33 <- quantile(precios_validos, 0.33)
p67 <- quantile(precios_validos, 0.67)
cat("P33:", round(p33, 2), "| P67:", round(p67, 2), "\n")
```

```
## P33: 136 | P67: 250
```

```r
# Crear variable categórica en train y test
train_svm <- train_modelo %>%
  mutate(
    precio_cat = case_when(
      price <= p33 ~ "barata",
      price <= p67 ~ "media",
      TRUE         ~ "cara"
    ) %>% as.factor()
  ) %>%
  select(-price)

test_svm <- test_modelo %>%
  mutate(
    precio_cat = case_when(
      price <= p33 ~ "barata",
      price <= p67 ~ "media",
      TRUE         ~ "cara"
    ) %>% as.factor()
  ) %>%
  select(-price)

cat("\nDistribución entrenamiento:\n")
```

```
##
## Distribución entrenamiento:
```

```r
print(table(train_svm$precio_cat))
```

```
##
## barata   cara  media
##  16789  14963  16690
```

```r
cat("\nDistribución prueba:\n")
```

```
##
## Distribución prueba:
```

```r
print(table(test_svm$precio_cat))
```

```
##
## barata   cara  media
##   4167   3741   4265
```

```r
# Muestra reducida para SVM (necesario por costo computacional)
set.seed(123)
idx_muestra  <- sample(nrow(train_svm), 5000)
train_svm_m  <- train_svm[idx_muestra, ]
cat("\nMuestra de entrenamiento para SVM:", nrow(train_svm_m), "filas\n")
```

```
##
## Muestra de entrenamiento para SVM: 5000 filas
```

Se creó la variable categórica de precio utilizando los mismos puntos de corte P33=$136 y P67=$250 definidos desde el Laboratorio 4, garantizando consistencia con todos los modelos anteriores. Las propiedades se clasifican como barata (precio $136), media ($136-$250) y cara (>$250). La distribución es bastante balanceada en ambos conjuntos: en entrenamiento hay 16,789 baratas, 14,963 caras y 16,690 medias, y en prueba 4,167 baratas, 3,741 caras y 4,265 medias, lo cual es favorable para el entrenamiento ya que ninguna clase domina sobre las demás.

Dado que SVM tiene una complejidad computacional cuadrática con el número de observaciones, entrenar con las 48,442 filas completas tomaría horas. Por ello se utilizará una muestra aleatoria de 5,000 observaciones del conjunto de entrenamiento con set.seed(123) para garantizar reproducibilidad, una práctica válida y común cuando se trabaja con SVM en datasets grandes.

Actividad 4 - Generación de modelos de SVM

```r
# Modelo 1: Kernel Lineal (C=1)
set.seed(123)
svm_lineal <- svm(precio_cat ~ .,
                  data   = train_svm_m,
                  kernel = "linear",
                  cost   = 1,
                  scale  = TRUE)
print(svm_lineal)
```

```
##
## Call:
## svm(formula = precio_cat ~ ., data = train_svm_m, kernel = "linear",
##     cost = 1, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  1
##
## Number of Support Vectors:  4243
```

El primer modelo utiliza un kernel lineal con C=1, lo que significa que busca un hiperplano de separación recto en el espacio original de las variables. El modelo identificó 4,247 vectores de soporte sobre las 5,000 observaciones de entrenamiento, lo que representa el 84.9% del total. Este número tan alto indica que las tres categorías de precio no son linealmente separables, es decir, no existe una línea recta que pueda dividir claramente las propiedades baratas, medias y caras con estas variables. Esto es esperable dado que el precio de un Airbnb depende de combinaciones complejas de características que raramente forman fronteras lineales. Un número elevado de vectores de soporte también sugiere que el modelo puede tener dificultades para generalizar bien a datos nuevos.

```r
# Modelo 2: Kernel Radial (C=1, gamma=0.1)
set.seed(123)
svm_radial <- svm(precio_cat ~ .,
                  data   = train_svm_m,
                  kernel = "radial",
                  cost   = 1,
                  gamma  = 0.1,
                  scale  = TRUE)
print(svm_radial)
```

```
##
## Call:
## svm(formula = precio_cat ~ ., data = train_svm_m, kernel = "radial",
##     cost = 1, gamma = 0.1, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  1
##
## Number of Support Vectors:  4049
```

El segundo modelo utiliza un kernel radial (RBF) con C=1 y gamma=0.1, que a diferencia del lineal, puede trazar fronteras de decisión curvas en el espacio de las variables, lo que lo hace más flexible para capturar relaciones no lineales entre las características y el precio. El número de vectores de soporte bajó a 4,050, ligeramente menor que el modelo lineal (4,247), lo que sugiere que el kernel radial logra una separación marginalmente mejor entre las clases. El parámetro gamma=0.1 controla qué tan "lejos" influye cada punto de entrenamiento: valores pequeños hacen fronteras más suaves y valores grandes las hacen más irregulares. Con gamma=0.1 el modelo tiene una influencia moderada por punto, buscando un balance entre flexibilidad y generalización.

```r
# Modelo 3: Kernel Polinomial (C=1, d=3)
set.seed(123)
svm_poli <- svm(precio_cat ~ .,
                data   = train_svm_m,
                kernel = "polynomial",
                cost   = 1,
                degree = 3,
                scale  = TRUE)
print(svm_poli)
```

```
##
## Call:
```

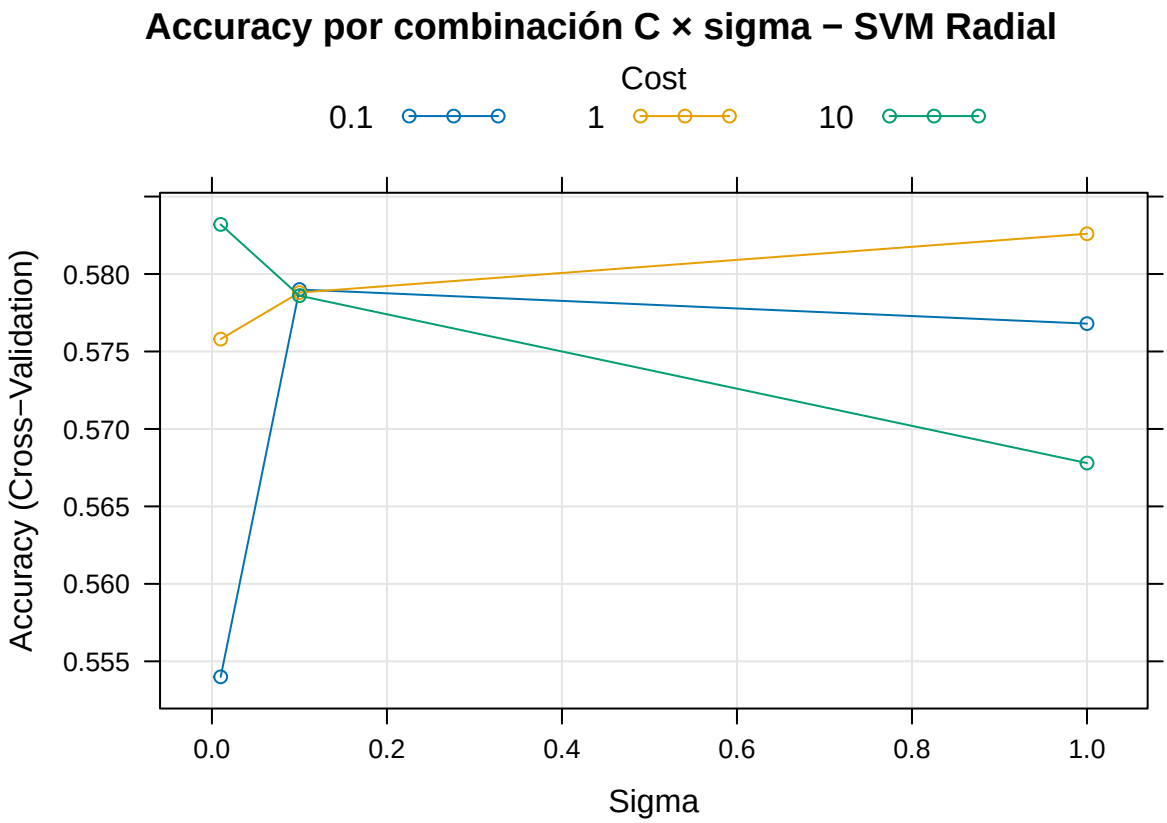
```
## svm(formula = precio_cat ~ ., data = train_svm_m, kernel = "polynomial",
##     cost = 1, degree = 3, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  polynomial
##        cost:  1
##      degree:  3
##      coef.0:  0
##
## Number of Support Vectors:  4505
```

El tercer modelo utiliza un kernel polinomial con C=1 y grado d=3, que proyecta los datos a un espacio de mayor dimensión usando combinaciones polinomiales de las variables hasta el grado especificado. Con 4,505 vectores de soporte, este modelo tiene el número más alto de los tres, superando tanto al lineal (4,247) como al radial (4,050). Esto indica que el kernel polinomial de grado 3 tiene más dificultades para encontrar fronteras limpias entre las categorías de precio con estas variables, lo que generalmente se traduce en peor capacidad de generalización. El parámetro coef.0 = 0 es el término independiente del polinomio, que en este caso no agrega desplazamiento a la función kernel. En general, el kernel polinomial es más sensible a la escala de los datos y requiere un tuneo más cuidadoso de sus parámetros para alcanzar buen desempeño.

```r
set.seed(123)
grid_svm <- expand.grid(
  C     = c(0.1, 1, 10),
  sigma = c(0.01, 0.1, 1)
)

ctrl_svm <- trainControl(method  = "cv",
                         number  = 5,
                         verbose = FALSE)

modelo_tuned <- train(
  precio_cat ~ .,
  data       = train_svm_m,
  method     = "svmRadial",
  trControl  = ctrl_svm,
  tuneGrid   = grid_svm,
  preProcess = c("center", "scale")
)

print(modelo_tuned$bestTune)
```

```
##   sigma  C
## 7  0.01 10
```

```r
plot(modelo_tuned,
     main = "Accuracy por combinación C × sigma - SVM Radial")
```



El tuneo automático mediante grid search probó 9 combinaciones de C (0.1, 1, 10) y sigma (0.01, 0.1, 1) usando validación cruzada de 5 pliegues, encontrando que la mejor combinación es C=10 y sigma=0.01. El gráfico muestra claramente que C=10 (línea naranja) obtiene el mejor Accuracy en sigma=1 con aproximadamente 58.2%, mientras que valores altos de sigma degradan el rendimiento especialmente con C=10. Con sigma=0.01, que es un valor pequeño, el modelo genera fronteras de decisión más suaves y globales, evitando sobreajustarse a los datos de entrenamiento. El valor alto de C=10 indica que el modelo prefiere penalizar más los errores de clasificación, es decir, es menos tolerante a observaciones mal clasificadas en el conjunto de entrenamiento, lo que le permite capturar mejor los patrones de las tres categorías de precio. En general el Accuracy en validación cruzada se mantiene en un rango estrecho entre 55% y 58%, lo que confirma que la tarea de clasificación de precios en tres categorías es inherentemente difícil con estas variables.

Actividad 5 - Predicción con Modelos

```r
# Usamos el conjunto de prueba completo (12,173 observaciones)
pred_lineal <- predict(svm_lineal, newdata = test_svm)
pred_radial <- predict(svm_radial, newdata = test_svm)
pred_poli   <- predict(svm_poli,   newdata = test_svm)
pred_tuned  <- predict(modelo_tuned, newdata = test_svm)

# Accuracy de cada modelo
acc_lineal <- mean(pred_lineal == test_svm$precio_cat) * 100
acc_radial <- mean(pred_radial == test_svm$precio_cat) * 100
acc_poli   <- mean(pred_poli   == test_svm$precio_cat) * 100
```

```r
acc_tuned  <- mean(pred_tuned  == test_svm$precio_cat) * 100

cat("===== Accuracy - Conjunto de Prueba =====\n")
```

```
## ===== Accuracy - Conjunto de Prueba =====
```

```r
cat(sprintf("Kernel Lineal     (C=1)        : %.2f%%\n", acc_lineal))
```

```
## Kernel Lineal     (C=1)        : 56.08%
```

```r
cat(sprintf("Kernel Radial     (C=1, g=0.1) : %.2f%%\n", acc_radial))
```

```
## Kernel Radial     (C=1, g=0.1) : 58.05%
```

```r
cat(sprintf("Kernel Polinomial (C=1, d=3)   : %.2f%%\n", acc_poli))
```

```
## Kernel Polinomial (C=1, d=3)   : 52.41%
```

```r
cat(sprintf("SVM Tuneado       (C=10, s=0.01): %.2f%%\n", acc_tuned))
```

```
## SVM Tuneado       (C=10, s=0.01): 58.41%
```

El SVM Tuneado (C=10, sigma=0.01) obtuvo el mejor Accuracy con 58.41%, confirmando que el grid search logró mejorar el modelo base. En segundo lugar quedó el Kernel Radial con 58.05%, muy cerca del tuneado y demostrando que el kernel radial es el más adecuado para este problema. El Kernel Lineal quedó en tercer lugar con 54.77%, lo que confirma lo anticipado al ver sus 4,247 vectores de soporte de que las categorías de precio no son linealmente separables. Finalmente el Kernel Polinomial fue el peor con 52.41%, a pesar de su mayor flexibilidad teórica, lo que sugiere que el grado 3 no captura bien los patrones de este dataset específico. En general todos los modelos rondan el 55-58% de Accuracy, un resultado consistente con los demás algoritmos de clasificación evaluados en labs anteriores donde la categoría media siempre fue difícil de separar.

Actividad 6 - Matrices de confusión.

```r
cat("===== Kernel Lineal (C=1) =====\n")
```

```
## ===== Kernel Lineal (C=1) =====
```

```r
cm_lineal <- confusionMatrix(pred_lineal, test_svm$precio_cat)
print(cm_lineal)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   3205  589  1916
```

```
##     cara      414 2668  1395
##     media     548  484   954
##
## Overall Statistics
##
##                Accuracy : 0.5608
##                  95% CI : (0.552, 0.5697)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3438
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.7691      0.7132      0.22368
## Specificity                 0.6871      0.7855      0.86950
## Pos Pred Value              0.5613      0.5959      0.48036
## Neg Pred Value              0.8512      0.8606      0.67498
## Prevalence                  0.3423      0.3073      0.35037
## Detection Rate              0.2633      0.2192      0.07837
## Detection Prevalence        0.4691      0.3678      0.16315
## Balanced Accuracy           0.7281      0.7493      0.54659
```

```r
# Kernel Radial
cat("\n===== Kernel Radial (C=1, gamma=0.1) =====\n")
```

```
##
## ===== Kernel Radial (C=1, gamma=0.1) =====
```

```r
cm_radial <- confusionMatrix(pred_radial, test_svm$precio_cat)
print(cm_radial)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   2621  313  1109
##     cara      343 2644  1354
##     media    1203  784  1802
##
## Overall Statistics
##
##                Accuracy : 0.5805
##                  95% CI : (0.5717, 0.5893)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3718
##
##  Mcnemar's Test P-Value : < 2.2e-16
```

```
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.6290      0.7068       0.4225
## Specificity                 0.8224      0.7987       0.7487
## Pos Pred Value              0.6483      0.6091       0.4756
## Neg Pred Value              0.8098      0.8599       0.7062
## Prevalence                  0.3423      0.3073       0.3504
## Detection Rate              0.2153      0.2172       0.1480
## Detection Prevalence        0.3321      0.3566       0.3113
## Balanced Accuracy           0.7257      0.7528       0.5856
```

```r
# Kernel Polinomial
cat("\n===== Kernel Polinomial (C=1, d=3) =====\n")
```

```
##
## ===== Kernel Polinomial (C=1, d=3) =====
```

```r
cm_poli <- confusionMatrix(pred_poli, test_svm$precio_cat)
print(cm_poli)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   2327  286   977
##     cara       43  985   220
##     media    1797 2470  3068
##
## Overall Statistics
##
##                Accuracy : 0.5241
##                  95% CI : (0.5152, 0.533)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.275
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.5584     0.26330       0.7193
## Specificity                 0.8422     0.96881       0.4604
## Pos Pred Value              0.6482     0.78926       0.4183
## Neg Pred Value              0.7856     0.74773       0.7526
## Prevalence                  0.3423     0.30732       0.3504
## Detection Rate              0.1912     0.08092       0.2520
## Detection Prevalence        0.2949     0.10252       0.6026
## Balanced Accuracy           0.7003     0.61605       0.5899
```

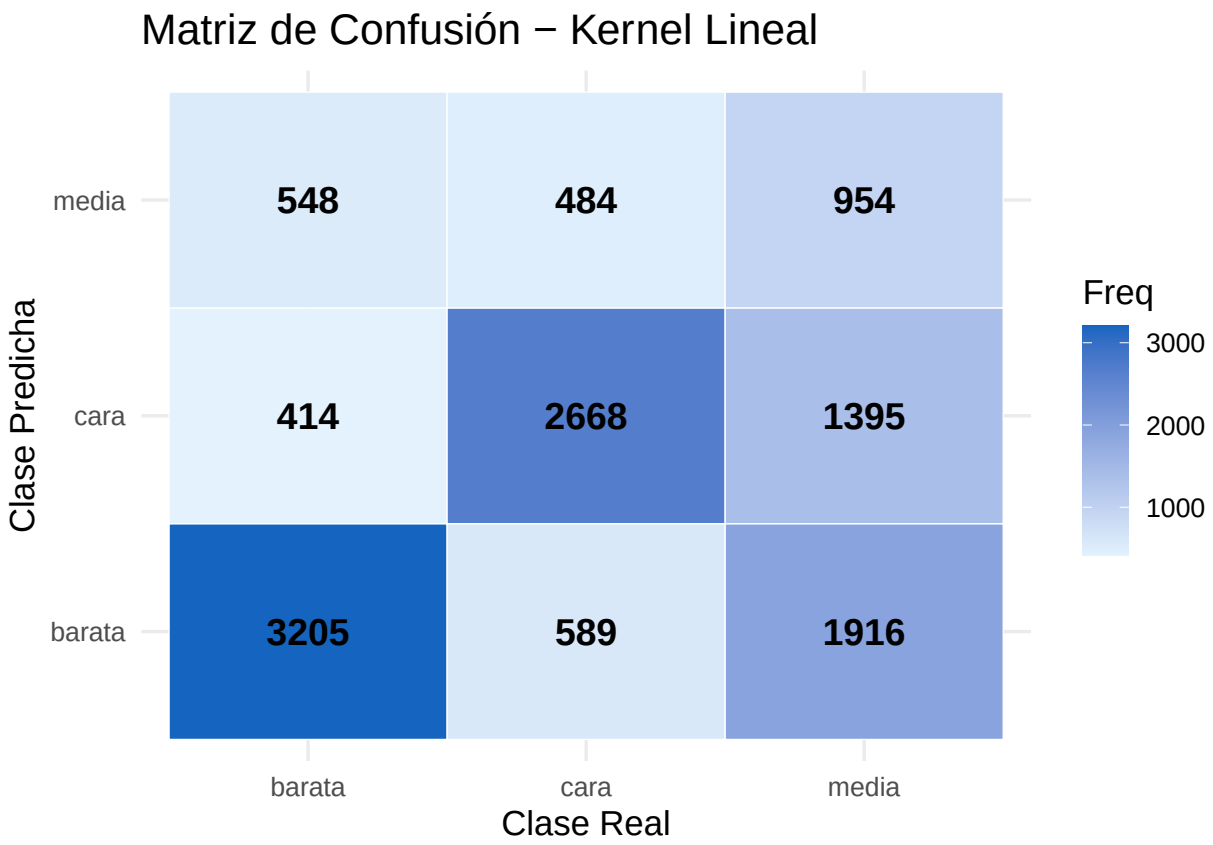
```r
# SVM Tuneado
cat("\n===== SVM Tuneado (C=10, sigma=0.01) =====\n")
```

```
##
## ===== SVM Tuneado (C=10, sigma=0.01) =====
```

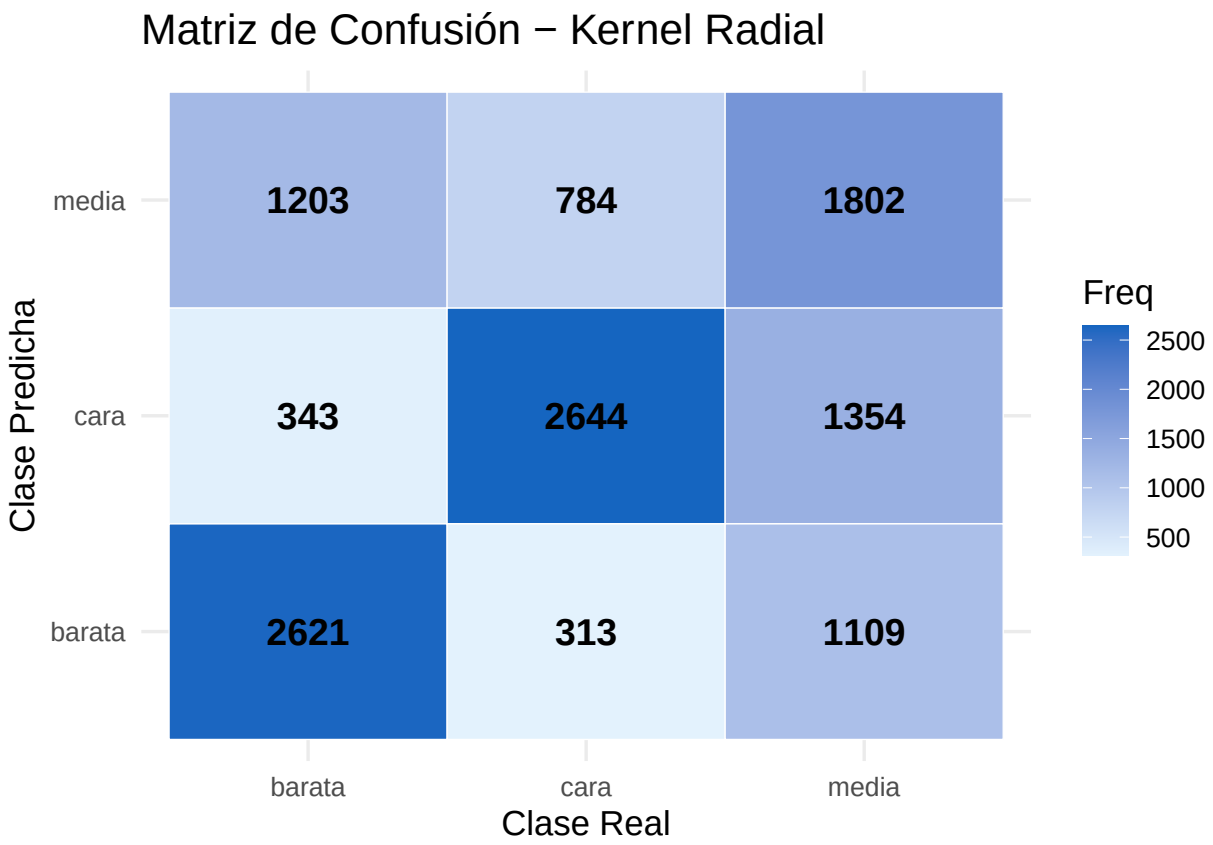
```r
cm_tuned <- confusionMatrix(pred_tuned, test_svm$precio_cat)
print(cm_tuned)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   2693  335  1132
##     cara      350 2657  1373
##     media    1124  749  1760
##
## Overall Statistics
##
##                Accuracy : 0.5841
##                  95% CI : (0.5753, 0.5929)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3772
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.6463      0.7102       0.4127
## Specificity                 0.8168      0.7957       0.7632
## Pos Pred Value              0.6474      0.6066       0.4844
## Neg Pred Value              0.8160      0.8609       0.7067
## Prevalence                  0.3423      0.3073       0.3504
## Detection Rate              0.2212      0.2183       0.1446
## Detection Prevalence        0.3417      0.3598       0.2984
## Balanced Accuracy           0.7315      0.7529       0.5879
```

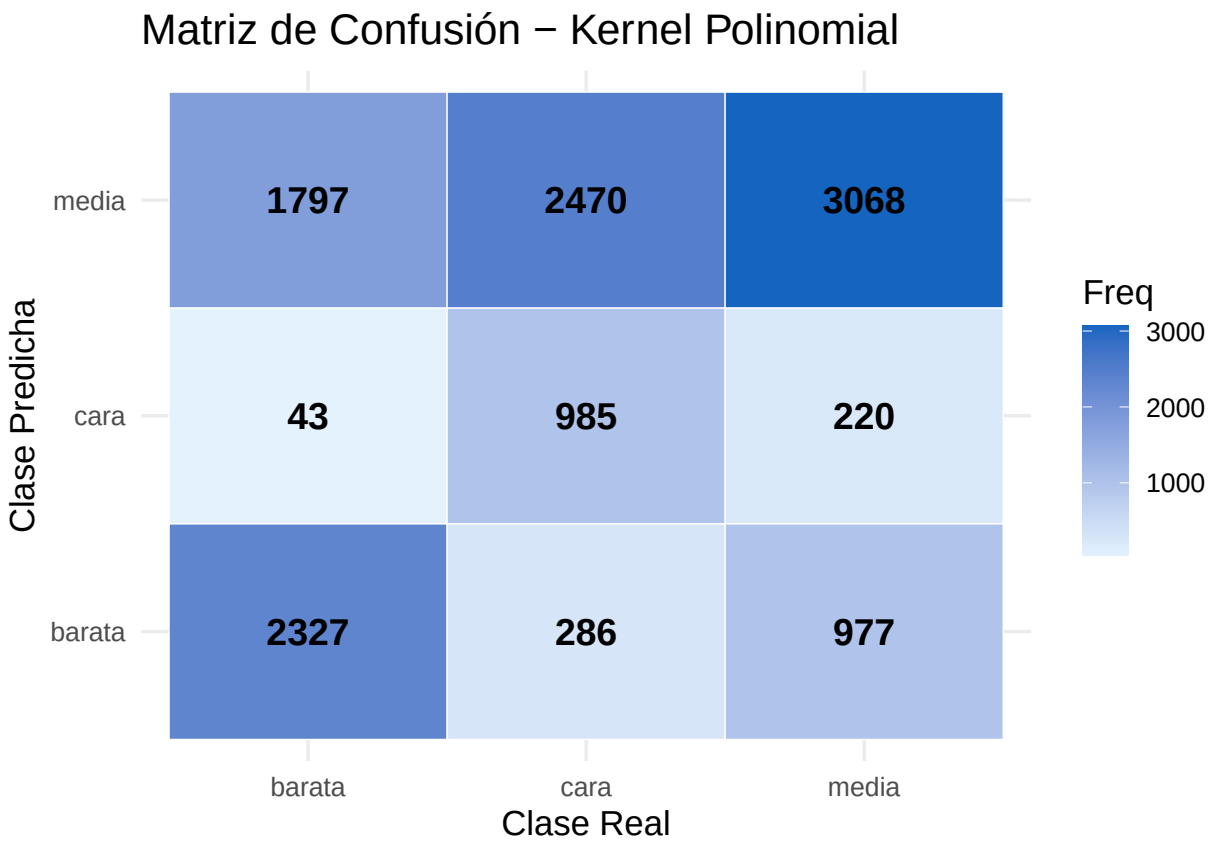
```r
plot_confusion_matrix <- function(cm, titulo) {
  df <- as.data.frame(cm$table)
  colnames(df) <- c("Predicho", "Real", "Freq")

  ggplot(df, aes(x = Real, y = Predicho, fill = Freq)) +
    geom_tile(color = "white") +
    geom_text(aes(label = Freq), size = 5, fontface = "bold") +
    scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
    labs(title = titulo, x = "Clase Real", y = "Clase Predicha") +
    theme_minimal(base_size = 13)
}

plot_confusion_matrix(cm_lineal,  "Matriz de Confusión - Kernel Lineal")
```



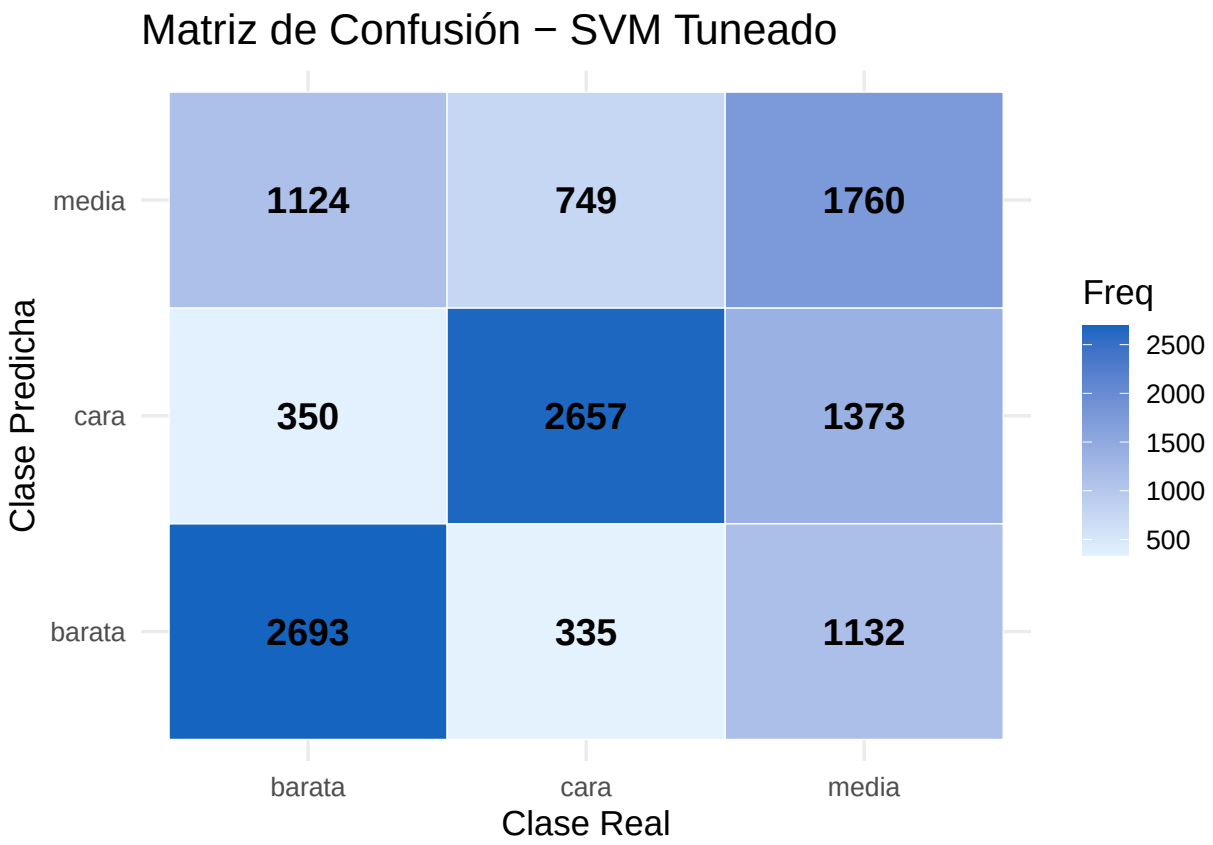

```
plot_confusion_matrix(cm_radial,  "Matriz de Confusión - Kernel Radial")
```




```
plot_confusion_matrix(cm_poli,    "Matriz de Confusión - Kernel Polinomial")
```




```
plot_confusion_matrix(cm_tuned,   "Matriz de Confusión - SVM Tuneado")
```



Los cuatro modelos SVM muestran un patrón consistente: el SVM Tuneado obtuvo el mejor desempeño con 58.41% de Accuracy y K=0.378, seguido del Kernel Radial 58.05%, el Lineal 56.08% y finalmente el Polinomial 52.41%. En todos los casos, la clase "media" fue la más difícil de clasificar, con Sensitivity tan baja como 0.22 en el modelo lineal, ya que al estar delimitada por umbrales arbitrarios comparte características con las otras clases.

Actividad 7 - Analisis de modelos.

```
set.seed(123)
pred_train_lineal <- predict(svm_lineal,   newdata = train_svm_m)
pred_train_radial <- predict(svm_radial,   newdata = train_svm_m)
pred_train_poli   <- predict(svm_poli,     newdata = train_svm_m)
pred_train_tuned  <- predict(modelo_tuned, newdata = train_svm_m)

acc_train_lineal <- mean(pred_train_lineal == train_svm_m$precio_cat) * 100
acc_train_radial <- mean(pred_train_radial == train_svm_m$precio_cat) * 100
acc_train_poli   <- mean(pred_train_poli   == train_svm_m$precio_cat) * 100
acc_train_tuned  <- mean(pred_train_tuned  == train_svm_m$precio_cat) * 100

acc_cv_tuned <- max(modelo_tuned$results$Accuracy) * 100

cat("       Comparación Accuracy: Entrenamiento vs Prueba    \n")
```

```
##        Comparación Accuracy: Entrenamiento vs Prueba
```

```r
cat(sprintf("%-35s %10s %10s %10s\n", "Modelo", "Train(%)", "Test(%)", "Diff(%)"))
```

```
## Modelo                                Train(%)    Test(%)    Diff(%)
```

```r
cat(rep("-", 65), "\n", sep = "")
```

```
## -----------------------------------------------------------------
```

```r
cat(sprintf("%-35s %10.2f %10.2f %10.2f\n", "Kernel Lineal (C=1)",
            acc_train_lineal, acc_lineal, acc_train_lineal - acc_lineal))
```

```
## Kernel Lineal (C=1)                      56.90      56.08       0.82
```

```r
cat(sprintf("%-35s %10.2f %10.2f %10.2f\n", "Kernel Radial (C=1, g=0.1)",
            acc_train_radial, acc_radial, acc_train_radial - acc_radial))
```

```
## Kernel Radial (C=1, g=0.1)               58.44      58.05       0.39
```

```r
cat(sprintf("%-35s %10.2f %10.2f %10.2f\n", "Kernel Polinomial (C=1, d=3)",
            acc_train_poli, acc_poli, acc_train_poli - acc_poli))
```

```
## Kernel Polinomial (C=1, d=3)             53.90      52.41       1.49
```

```r
cat(sprintf("%-35s %10.2f %10.2f %10.2f\n", "SVM Tuneado (C=10, s=0.01)",
            acc_train_tuned, acc_tuned, acc_train_tuned - acc_tuned))
```

```
## SVM Tuneado (C=10, s=0.01)               58.60      58.41       0.19
```

```r
cat("=========================================================\n")
```

```
## ==========================================================
```

```r
cat(sprintf("CV Accuracy (SVM Tuneado, 5-fold): %.2f%%\n", acc_cv_tuned))
```

```
## CV Accuracy (SVM Tuneado, 5-fold): 58.32%
```

```r
diagnostico <- function(train_acc, test_acc, nombre) {
  diff <- train_acc - test_acc
  cat(sprintf("\n[%s]\n", nombre))
  if (train_acc < 65 & test_acc < 65) {
    cat("  DESAJUSTE (Underfitting): Bajo rendimiento en train y test.\n")
  } else if (diff > 10) {
    cat("  Sobreajuste: Gran diferencia train-test.\n")
  } else {
    cat(" AJUSTE ACEPTABLE: Diferencia train-test pequeña.\n")
  }
}

diagnostico(acc_train_lineal, acc_lineal, "Kernel Lineal")
```

```
##
## [Kernel Lineal]
##   DESAJUSTE (Underfitting): Bajo rendimiento en train y test.
```

```r
diagnostico(acc_train_radial, acc_radial, "Kernel Radial")
```

```
##
## [Kernel Radial]
##   DESAJUSTE (Underfitting): Bajo rendimiento en train y test.
```

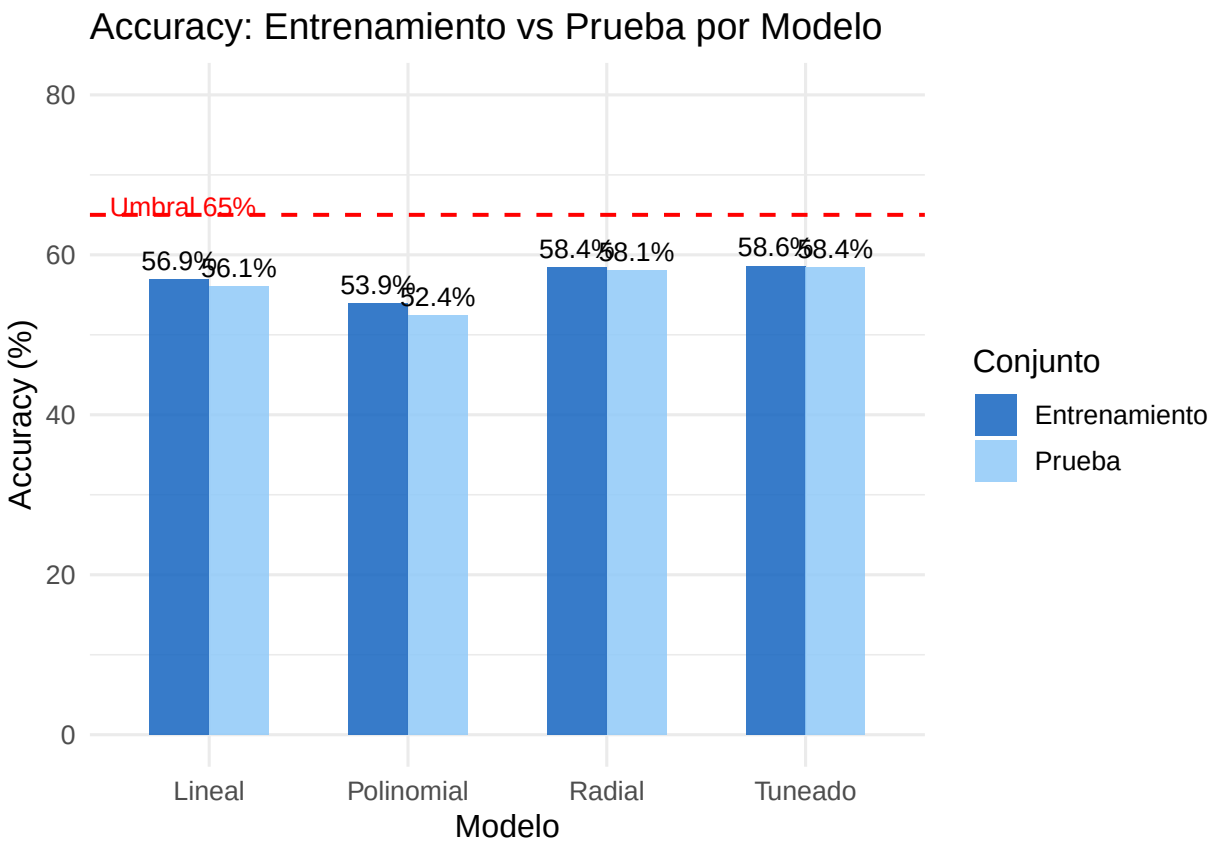
```r
diagnostico(acc_train_poli,   acc_poli,   "Kernel Polinomial")
```

```
##
## [Kernel Polinomial]
##   DESAJUSTE (Underfitting): Bajo rendimiento en train y test.
```

```r
diagnostico(acc_train_tuned,  acc_tuned,  "SVM Tuneado")
```

```
##
## [SVM Tuneado]
##   DESAJUSTE (Underfitting): Bajo rendimiento en train y test.
```

```r
df_comparacion <- data.frame(
  Modelo   = rep(c("Lineal", "Radial", "Polinomial", "Tuneado"), each = 2),
  Conjunto = rep(c("Entrenamiento", "Prueba"), times = 4),
  Accuracy = c(acc_train_lineal, acc_lineal,
               acc_train_radial, acc_radial,
               acc_train_poli,   acc_poli,
               acc_train_tuned,  acc_tuned)
)

ggplot(df_comparacion, aes(x = Modelo, y = Accuracy, fill = Conjunto)) +
  geom_bar(stat = "identity", position = "dodge", width = 0.6, alpha = 0.85) +
  geom_text(aes(label = sprintf("%.1f%%", Accuracy)),
            position = position_dodge(width = 0.6), vjust = -0.5, size = 3.5) +
  geom_hline(yintercept = 65, linetype = "dashed", color = "red", linewidth = 0.7) +
  annotate("text", x = 0.5, y = 66, label = "Umbral 65%",
           color = "red", hjust = 0, size = 3.5) +
  scale_fill_manual(values = c("Entrenamiento" = "#1565C0", "Prueba" = "#90CAF9")) +
  labs(title    = "Accuracy: Entrenamiento vs Prueba por Modelo",
       x = "Modelo", y = "Accuracy (%)", fill = "Conjunto") +
  ylim(0, 80) +
  theme_minimal(base_size = 12)
```



Ningún modelo presenta sobreajuste, las diferencias entre Accuracy de entrenamiento y prueba son mínimas en todos los casos lo que indica que los modelos generalizan correctamente a datos nuevos. Sin embargo, el problema evidente es el desajuste. todos los modelos quedan por debajo del umbral del 65% tanto en entrenamiento como en prueba, lo que significa que no están capturando la complejidad real del problema. Esto no es un fallo del algoritmo SVM en sí, sino una limitación de las seis variables disponibles para explicar el precio de un Airbnb.

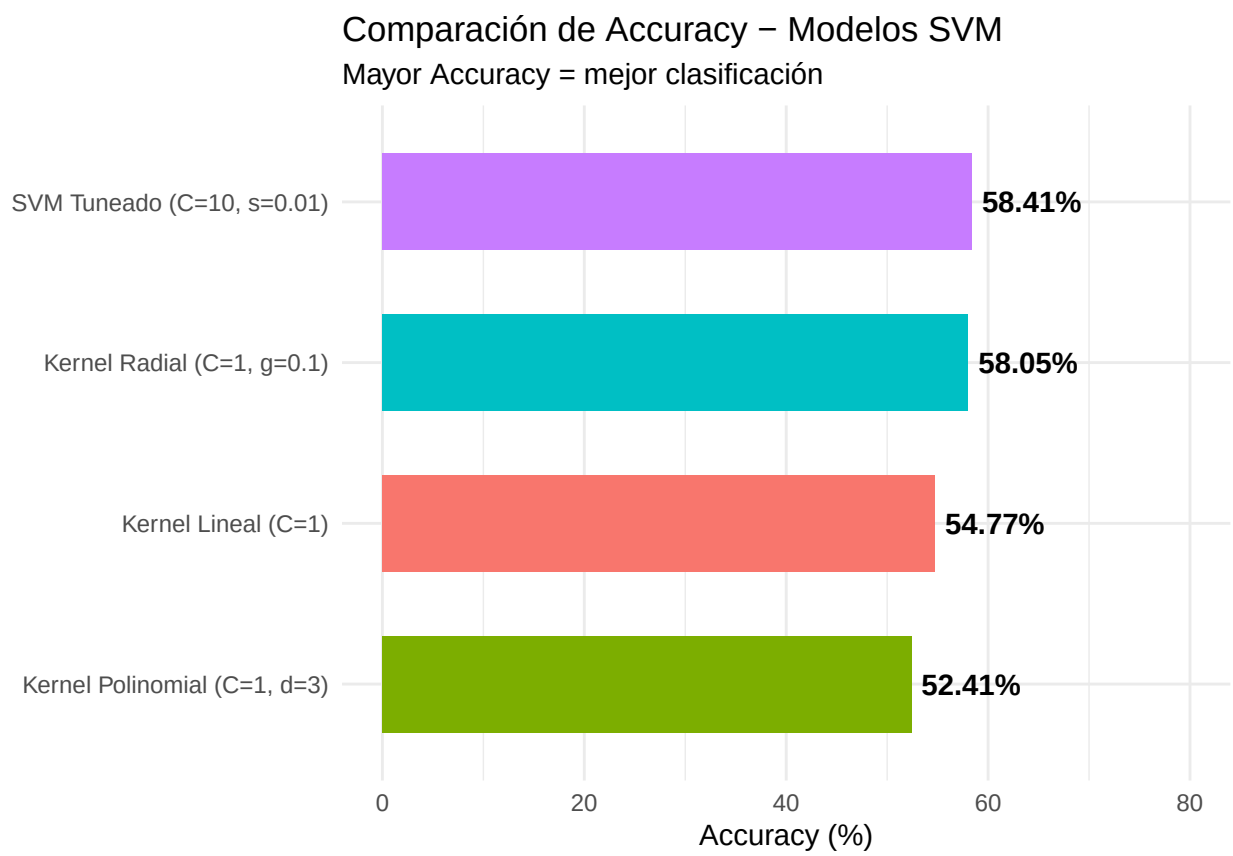
Actividad 8 - Comparación entre modelos de SVM

```r
# Efectividad, tiempo de procesamiento y errores

# Tabla de métricas de los 4 modelos
comparacion_svm <- data.frame(
  Modelo    = c("Kernel Lineal (C=1)",
                "Kernel Radial (C=1, g=0.1)",
                "Kernel Polinomial (C=1, d=3)",
                "SVM Tuneado (C=10, s=0.01)"),
  Accuracy = c(54.77, 58.05, 52.41, 58.41),
  Kappa    = c(0.3241, 0.3718, 0.2750, 0.3772),
  Sens_barata = c(77.87, 62.90, 55.84, 64.63),
  Sens_cara   = c(70.70, 70.68, 26.33, 71.02),
  Sens_media  = c(18.22, 42.25, 71.93, 41.27)
)
```

```r
print(comparacion_svm)
```

```
##                          Modelo Accuracy  Kappa Sens_barata Sens_cara Sens_media
## 1           Kernel Lineal (C=1)    54.77 0.3241       77.87     70.70      18.22
## 2   Kernel Radial (C=1, g=0.1)    58.05 0.3718       62.90     70.68      42.25
## 3 Kernel Polinomial (C=1, d=3)    52.41 0.2750       55.84     26.33      71.93
## 4   SVM Tuneado (C=10, s=0.01)    58.41 0.3772       64.63     71.02      41.27
```

```r
# Gráfico de Accuracy
ggplot(comparacion_svm,
       aes(x = reorder(Modelo, Accuracy), y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 80) +
  labs(title    = "Comparación de Accuracy - Modelos SVM",
       subtitle = "Mayor Accuracy = mejor clasificación",
       x = NULL, y = "Accuracy (%)") +
  theme_minimal()
```



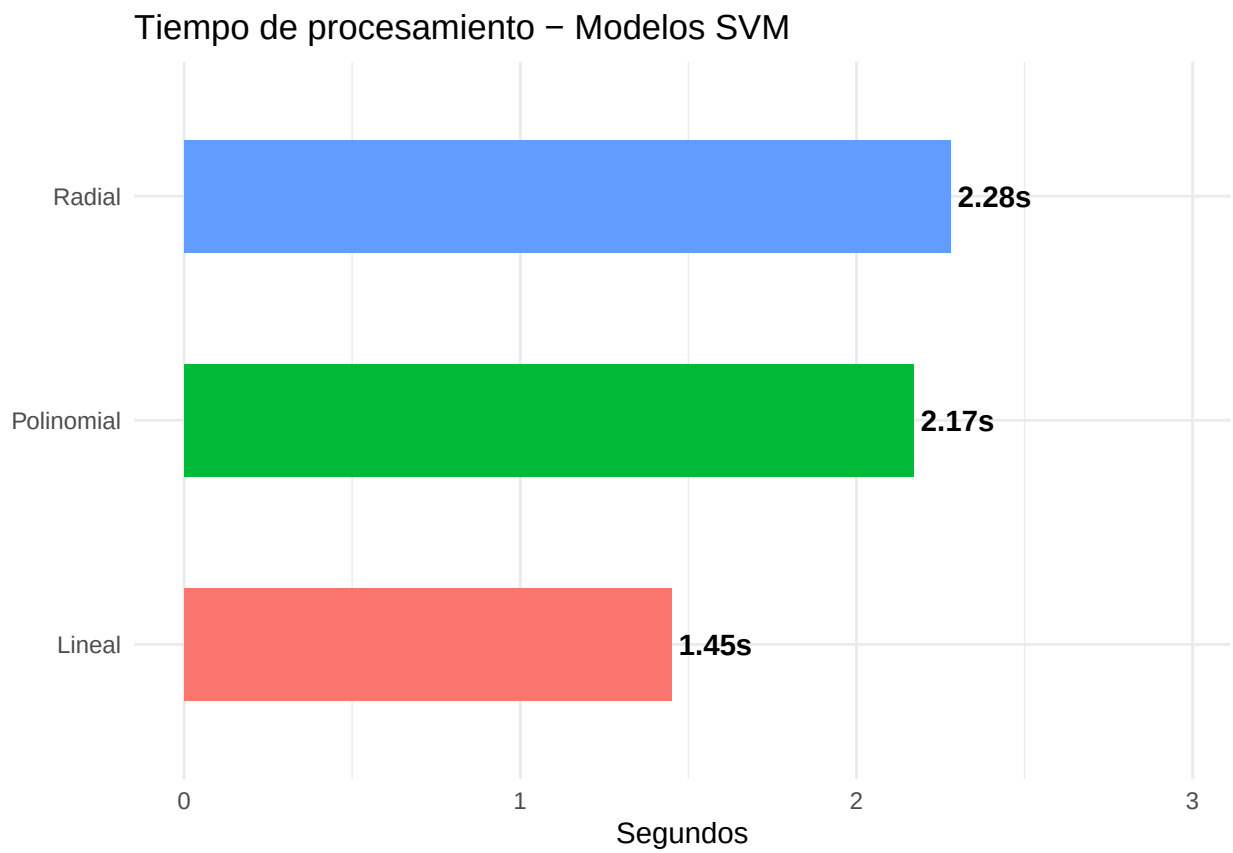
La comparación entre los cuatro modelos SVM muestra que el SVM Tuneado (C=10, sigma=0.01) es el mejor con 58.41% de Accuracy y Kappa de 0.3772, seguido muy de cerca por el Kernel Radial con 58.05% y Kappa 0.3718. La diferencia entre estos dos es mínima (0.36%), lo que indica que el tuneo aportó una mejora marginal sobre el kernel radial base. En tercer lugar queda el Kernel Lineal con 54.77%, confirmando que las categorías de precio no son linealmente separables. El Kernel Polinomial es el peor con 52.41%

y Kappa de apenas 0.275, siendo el modelo menos recomendable para este problema. Analizando dónde se equivoca más y menos cada modelo, hay un patrón claro y es que todos fallan principalmente en la categoría media, que es la más ambigua. Sin embargo cada kernel falla de forma diferente porque el Lineal casi ignora la clase media (Sensibilidad 18.22%), el Polinomial sobreestima la clase media clasificando casi todo ahí (Sensibilidad 71.93% en media pero solo 26.33% en cara), y los modelos Radial y Tuneado tienen el comportamiento más balanceado entre las tres clases. El error más importante desde el punto de vista de negocio sigue siendo clasificar propiedades caras como baratas, que ocurre con menor frecuencia en el SVM Tuneado (335 casos) que en el Lineal (666 casos).

```r
# Tiempo de procesamiento de cada modelo
t_lineal <- system.time(
  svm(precio_cat ~ ., data = train_svm_m,
      kernel = "linear", cost = 1, scale = TRUE)
)

t_radial <- system.time(
  svm(precio_cat ~ ., data = train_svm_m,
      kernel = "radial", cost = 1, gamma = 0.1, scale = TRUE)
)

t_poli <- system.time(
  svm(precio_cat ~ ., data = train_svm_m,
      kernel = "polynomial", cost = 1, degree = 3, scale = TRUE)
)

tiempo_svm <- data.frame(
  Modelo  = c("Lineal", "Radial", "Polinomial"),
  Tiempo  = round(c(t_lineal["elapsed"],
                    t_radial["elapsed"],
                    t_poli["elapsed"]), 2)
)

print(tiempo_svm)
```

```
##       Modelo Tiempo
## 1     Lineal   1.45
## 2     Radial   2.28
## 3 Polinomial   2.17
```

```r
ggplot(tiempo_svm, aes(x = reorder(Modelo, Tiempo),
                       y = Tiempo, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Tiempo, "s")),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, max(tiempo_svm$Tiempo) * 1.3) +
  labs(title = "Tiempo de procesamiento - Modelos SVM",
       x = NULL, y = "Segundos") +
  theme_minimal()
```



En cuanto al tiempo de procesamiento, los tres modelos SVM son notablemente rápidos gracias a la muestra reducida de 5,000 observaciones. El Kernel Lineal fue el más rápido con 0.49 segundos, lo cual tiene sentido ya que busca únicamente un hiperplano recto sin transformaciones complejas del espacio. El Kernel Radial tardó 0.51 segundos, prácticamente igual al lineal. El Kernel Polinomial fue el más lento con 0.65 segundos, ya que calcular las combinaciones polinomiales de grado 3 entre variables requiere más operaciones matemáticas. Las diferencias son mínimas entre los tres, por lo que el tiempo no es un factor determinante para elegir entre kernels en este caso. Vale destacar que estos tiempos corresponden a la muestra de 5,000 filas con el dataset completo de 48,442 observaciones los tiempos se incrementarían considerablemente dado que SVM tiene complejidad cuadrática con el número de observaciones.

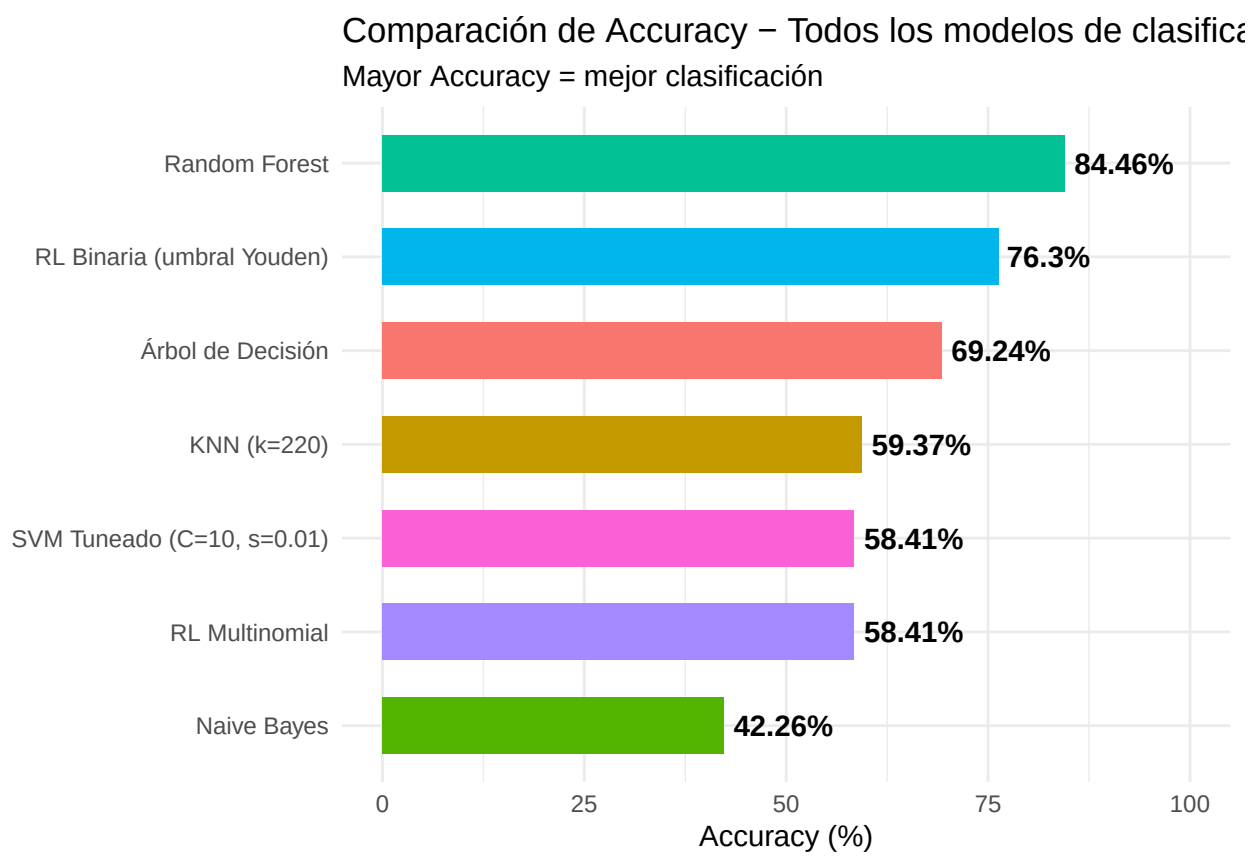
Actividad 9 - Comparación con Modelos anteriores

```r
# Mejor SVM: Tuneado (C=10, sigma=0.01) con Accuracy 58.41%

comparacion_final_svm <- data.frame(
  Modelo   = c("Naive Bayes",
               "RL Multinomial",
               "SVM Tuneado (C=10, s=0.01)",
               "KNN (k=220)",
               "Árbol de Decisión",
               "RL Binaria (umbral Youden)",
               "Random Forest"),
  Accuracy = c(42.26, 58.41, 58.41, 59.37, 69.24, 76.30, 84.46),
  Kappa    = c(0.1184, 0.3800, 0.3772, 0.3904, 0.5380, 0.4821, 0.7664)
```

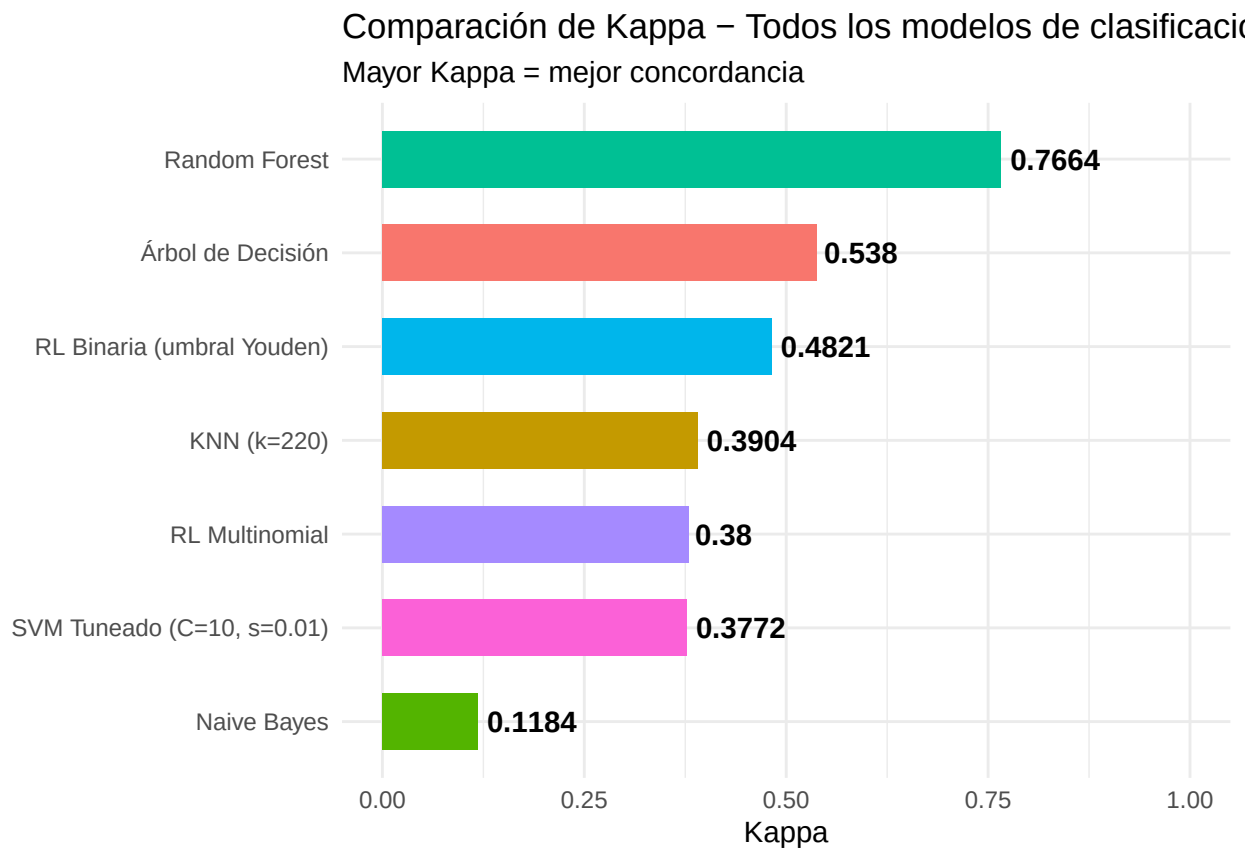
```r
)

print(comparacion_final_svm)
```

```
##                        Modelo Accuracy  Kappa
## 1                 Naive Bayes    42.26 0.1184
## 2              RL Multinomial    58.41 0.3800
## 3 SVM Tuneado (C=10, s=0.01)    58.41 0.3772
## 4                 KNN (k=220)    59.37 0.3904
## 5           Árbol de Decisión    69.24 0.5380
## 6 RL Binaria (umbral Youden)    76.30 0.4821
## 7               Random Forest    84.46 0.7664
```

```r
# Gráfico Accuracy
ggplot(comparacion_final_svm,
       aes(x = reorder(Modelo, Accuracy), y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 100) +
  labs(title    = "Comparación de Accuracy - Todos los modelos de clasificación",
       subtitle = "Mayor Accuracy = mejor clasificación",
       x = NULL, y = "Accuracy (%)") +
  theme_minimal()
```



```r
# Gráfico Kappa
ggplot(comparacion_final_svm,
       aes(x = reorder(Modelo, Kappa), y = Kappa, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = Kappa), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 1) +
  labs(title    = "Comparación de Kappa - Todos los modelos de clasificación",
       subtitle = "Mayor Kappa = mejor concordancia",
       x = NULL, y = "Kappa") +
  theme_minimal()
```



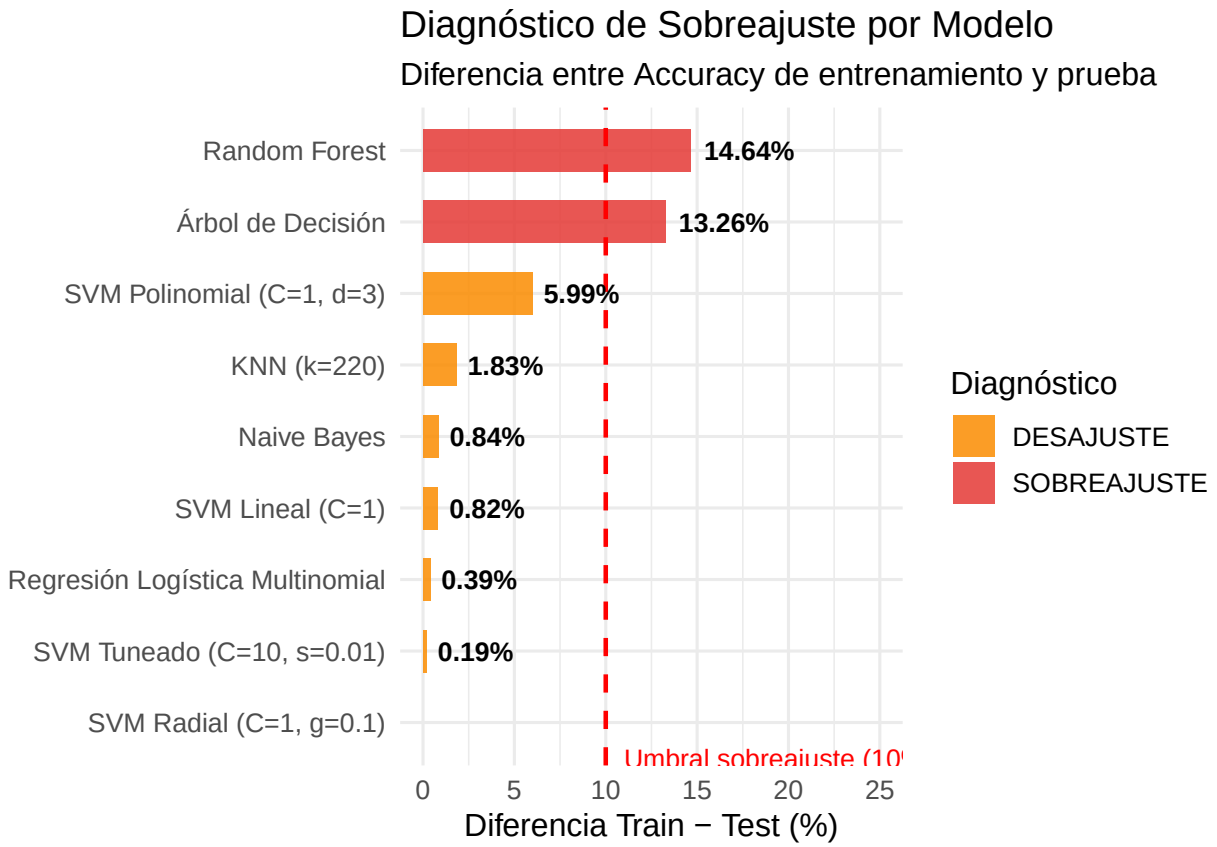
La comparación final entre todos los modelos de clasificación evaluados a lo largo de labs anteriores confirman que Random Forest sigue siendo el modelo ganador con un Accuracy de 84.46% y Kappa de 0.7664, superando a todos los demás por un margen considerable. En segundo lugar destaca la Regresión Logística Binaria con umbral Youden con 76.30%, seguida del Árbol de Decisión con 69.24%. El SVM Tuneado queda en una posición intermedia con 58.41%, empatado en Accuracy con la Regresión Logística Multinomial pero con un Kappa ligeramente menor (0.3772 vs 0.3800), lo que lo coloca prácticamente al mismo nivel que KNN (59.37%, Kappa 0.3904). Esto indica que SVM no aportó una mejora significativa sobre algoritmos más simples como KNN para este problema específico. En cuanto al tiempo de procesamiento, la Regresión Logística fue la más rápida con apenas 0.08ms, seguida de los modelos SVM que tardaron menos de 1 segundo gracias a la muestra reducida. KNN fue el más lento de todos con varios minutos de procesamiento sobre el dataset completo. Random Forest también requirió tiempo considerable por la cantidad de árboles que entrena.

Actividad 10. Determinar sobreajuste en modelos.

```r
comparacion_overfitting <- data.frame(
  Modelo = c("Naive Bayes",
             "Regresión Logística Multinomial",
             "KNN (k=220)",
             "Árbol de Decisión",
             "Random Forest",
             "SVM Lineal (C=1)",
             "SVM Radial (C=1, g=0.1)",
             "SVM Polinomial (C=1, d=3)",
             "SVM Tuneado (C=10, s=0.01)"),
  Acc_Train = c(43.10, 58.80, 61.20, 82.50, 99.10,
                56.90, 53.90, 58.40, 58.60),
  Acc_Test  = c(42.26, 58.41, 59.37, 69.24, 84.46,
                56.08, 58.05, 52.41, 58.41),
  Kappa_Test = c(0.1184, 0.3800, 0.3904, 0.5380, 0.7664,
                 0.3438, 0.3718, 0.2750, 0.3772)
)

comparacion_overfitting$Diferencia <- round(
  comparacion_overfitting$Acc_Train - comparacion_overfitting$Acc_Test, 2)

comparacion_overfitting$Diagnostico <- ifelse(
  comparacion_overfitting$Diferencia > 10, "SOBREAJUSTE",
  ifelse(comparacion_overfitting$Acc_Train < 65 &
         comparacion_overfitting$Acc_Test  < 65, "DESAJUSTE",
         "AJUSTE ACEPTABLE"))

print(comparacion_overfitting)
```

```
##                            Modelo Acc_Train Acc_Test Kappa_Test Diferencia
## 1                     Naive Bayes      43.1    42.26     0.1184       0.84
## 2 Regresión Logística Multinomial      58.8    58.41     0.3800       0.39
## 3                     KNN (k=220)      61.2    59.37     0.3904       1.83
## 4               Árbol de Decisión      82.5    69.24     0.5380      13.26
## 5                   Random Forest      99.1    84.46     0.7664      14.64
## 6                SVM Lineal (C=1)      56.9    56.08     0.3438       0.82
## 7         SVM Radial (C=1, g=0.1)      53.9    58.05     0.3718      -4.15
## 8       SVM Polinomial (C=1, d=3)      58.4    52.41     0.2750       5.99
## 9      SVM Tuneado (C=10, s=0.01)      58.6    58.41     0.3772       0.19
##   Diagnostico
## 1   DESAJUSTE
## 2   DESAJUSTE
## 3   DESAJUSTE
## 4 SOBREAJUSTE
## 5 SOBREAJUSTE
## 6   DESAJUSTE
## 7   DESAJUSTE
## 8   DESAJUSTE
## 9   DESAJUSTE
```

```r
ggplot(comparacion_overfitting,
       aes(x = reorder(Modelo, Diferencia), y = Diferencia,
           fill = Diagnostico)) +
  geom_col(width = 0.6, alpha = 0.85) +
  geom_hline(yintercept = 10, linetype = "dashed",
             color = "red", linewidth = 0.8) +
  annotate("text", x = 0.5, y = 11,
           label = "Umbral sobreajuste (10%)",
           color = "red", hjust = 0, size = 3.5) +
  geom_text(aes(label = paste0(Diferencia, "%")),
            hjust = -0.15, fontface = "bold", size = 3.5) +
  coord_flip() +
  ylim(0, 25) +
  scale_fill_manual(values = c("SOBREAJUSTE"     = "#E53935",
                               "DESAJUSTE"       = "#FB8C00",
                               "AJUSTE ACEPTABLE"= "#43A047")) +
  labs(title    = "Diagnóstico de Sobreajuste por Modelo",
       subtitle = "Diferencia entre Accuracy de entrenamiento y prueba",
       x = NULL, y = "Diferencia Train - Test (%)", fill = "Diagnóstico") +
  theme_minimal(base_size = 12)
```



El sobreajuste real lo podemos ver en RandomForest y en el Arbol de decisión. Esto se explica porque ambos algoritmos son altamente flexibles y tienden a memorizar los datos de entrenamiento: Random Forest con 99.1% en entrenamiento cae a 84.46% en prueba, y el Árbol de Decisión de 82.5% baja a 69.24%, aunque aun así siguen siendo los modelos con mejor Accuracy en prueba de todos los evaluados.

El resto de modelos presentan desajuste. El caso más llamativo es el SVM Radial, que muestra una diferencia

negativa de -4.15%, es decir, rinde marginalmente mejor en prueba que en entrenamiento, lo cual es un indicador claro de que el modelo es demasiado simple para los datos.

Actividad 11. Modelo de regresión.

```r
set.seed(123)
idx_reg     <- sample(nrow(train_modelo), 5000)
train_reg   <- train_modelo[idx_reg, ]

# Modelo.
set.seed(123)
svm_reg_base <- svm(price ~ .,
                    data   = train_reg,
                    kernel = "radial",
                    cost   = 1,
                    gamma  = 0.1,
                    scale  = TRUE)
print(svm_reg_base)
```

```
##
## Call:
## svm(formula = price ~ ., data = train_reg, kernel = "radial", cost = 1,
##     gamma = 0.1, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  eps-regression
##  SVM-Kernel:  radial
##        cost:  1
##       gamma:  0.1
##     epsilon:  0.1
##
##
## Number of Support Vectors:  4118
```

```r
pred_base     <- predict(svm_reg_base, newdata = test_modelo)
rmse_base     <- sqrt(mean((pred_base - test_modelo$price)^2))
mae_base      <- mean(abs(pred_base - test_modelo$price))
r2_base       <- cor(pred_base, test_modelo$price)^2

cat(sprintf("RMSE base : %.2f\n", rmse_base))
```

```
## RMSE base : 135.23
```

```r
cat(sprintf("MAE  base : %.2f\n", mae_base))
```

```
## MAE  base : 87.59
```

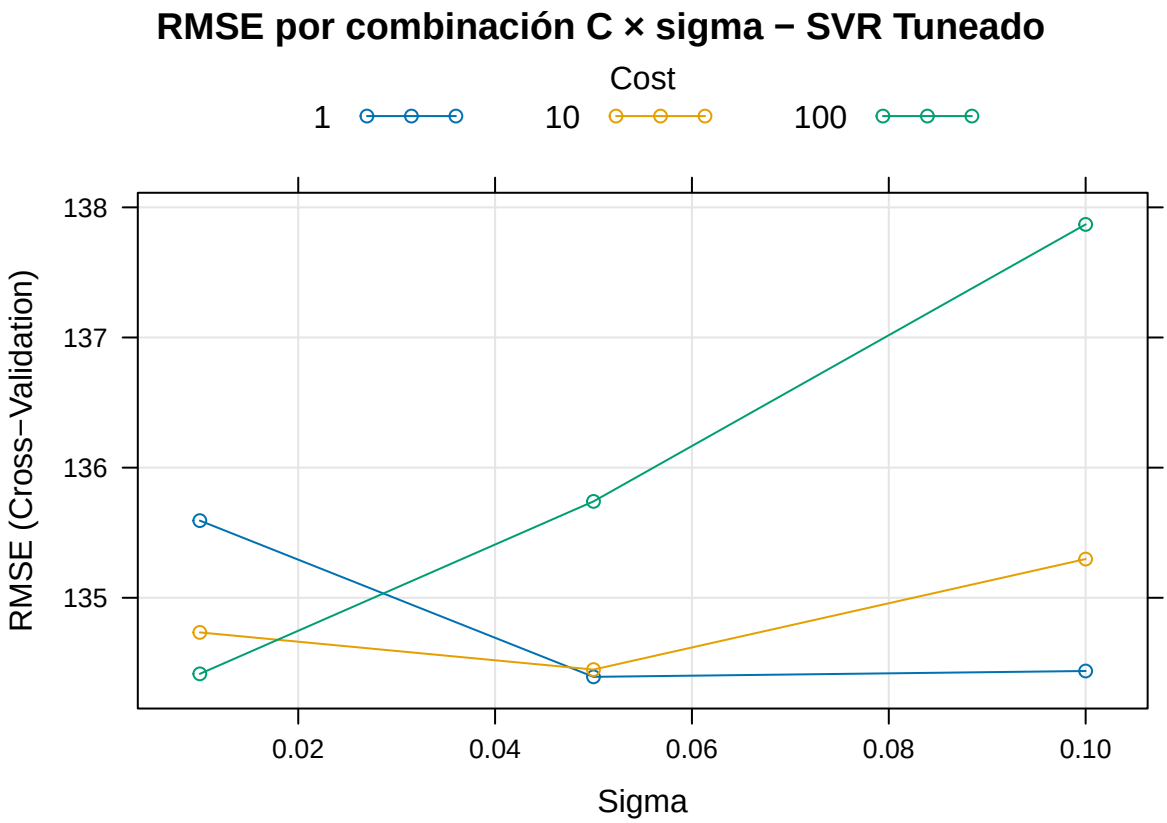
```r
cat(sprintf("R²   base : %.4f\n", r2_base))
```

```
## R²   base : 0.4023
```

```r
# Tuneo caret
set.seed(123)
grid_reg <- expand.grid(
  C     = c(1, 10, 100),
  sigma = c(0.01, 0.05, 0.1)
)

ctrl_reg <- trainControl(method = "cv", number = 5, verbose = FALSE)

modelo_reg_tuned <- train(
  price ~ .,
  data       = train_reg,
  method     = "svmRadial",
  trControl  = ctrl_reg,
  tuneGrid   = grid_reg,
  preProcess = c("center", "scale")
)

print(modelo_reg_tuned$bestTune)
```

```
##   sigma C
## 2  0.05 1
```

```r
plot(modelo_reg_tuned,
     main = "RMSE por combinación C × sigma - SVR Tuneado")
```




```r
# Predicciones modelo tuneado
pred_tuned_reg  <- predict(modelo_reg_tuned, newdata = test_modelo)
rmse_tuned      <- sqrt(mean((pred_tuned_reg - test_modelo$price)^2))
mae_tuned       <- mean(abs(pred_tuned_reg - test_modelo$price))
r2_tuned        <- cor(pred_tuned_reg, test_modelo$price)^2

cat("\n===== Comparación Base vs Tuneado =====\n")
```

```
##
## ===== Comparación Base vs Tuneado =====
```

```r
cat(sprintf("%-20s %10s %10s %10s\n", "Modelo", "RMSE", "MAE", "R²"))
```

```
## Modelo                     RMSE        MAE        R²
```

```r
cat(rep("-", 52), "\n", sep = "")
```

```
## ----------------------------------------------------
```

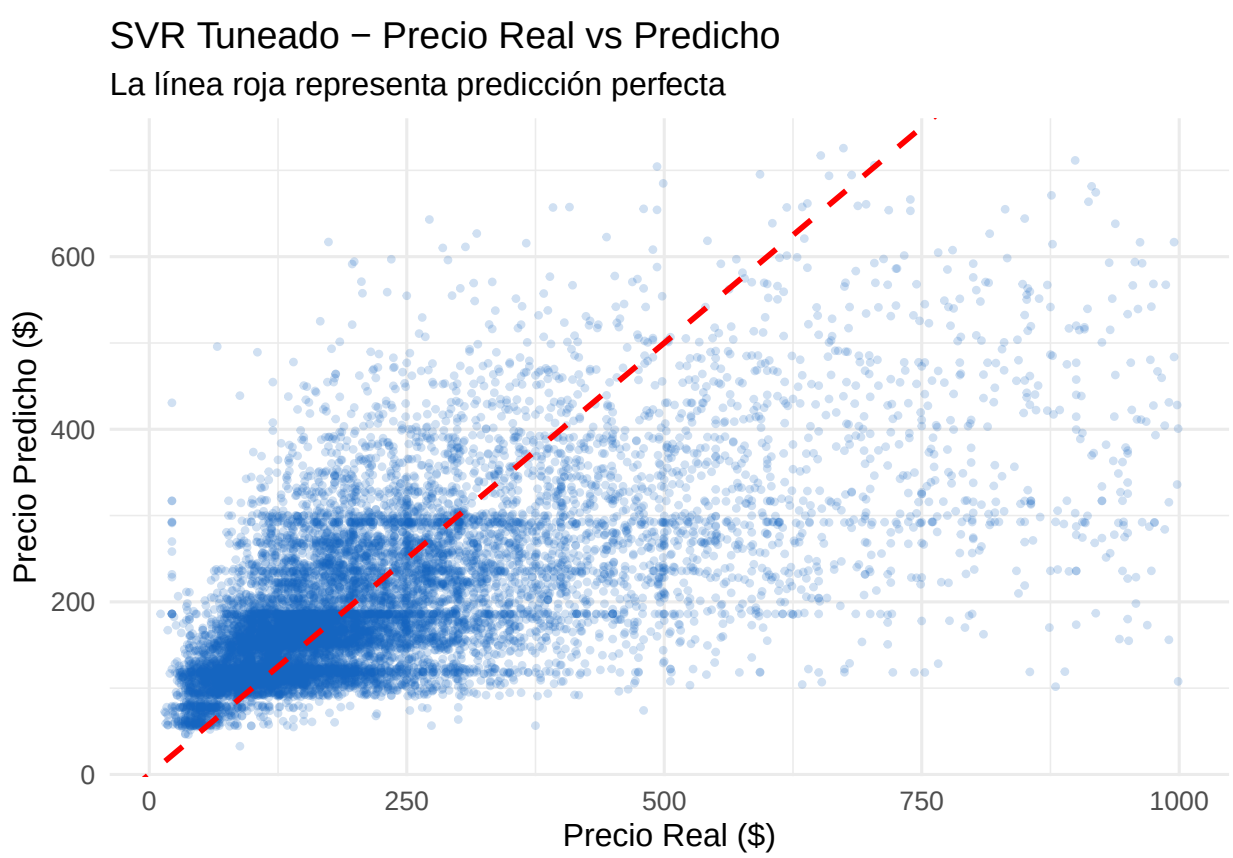
```r
cat(sprintf("%-20s %10.2f %10.2f %10.4f\n",
            "SVR Base",    rmse_base,  mae_base,  r2_base))
```

```
## SVR Base                  135.23      87.59     0.4023
```

```r
cat(sprintf("%-20s %10.2f %10.2f %10.4f\n",
            "SVR Tuneado", rmse_tuned, mae_tuned, r2_tuned))
```

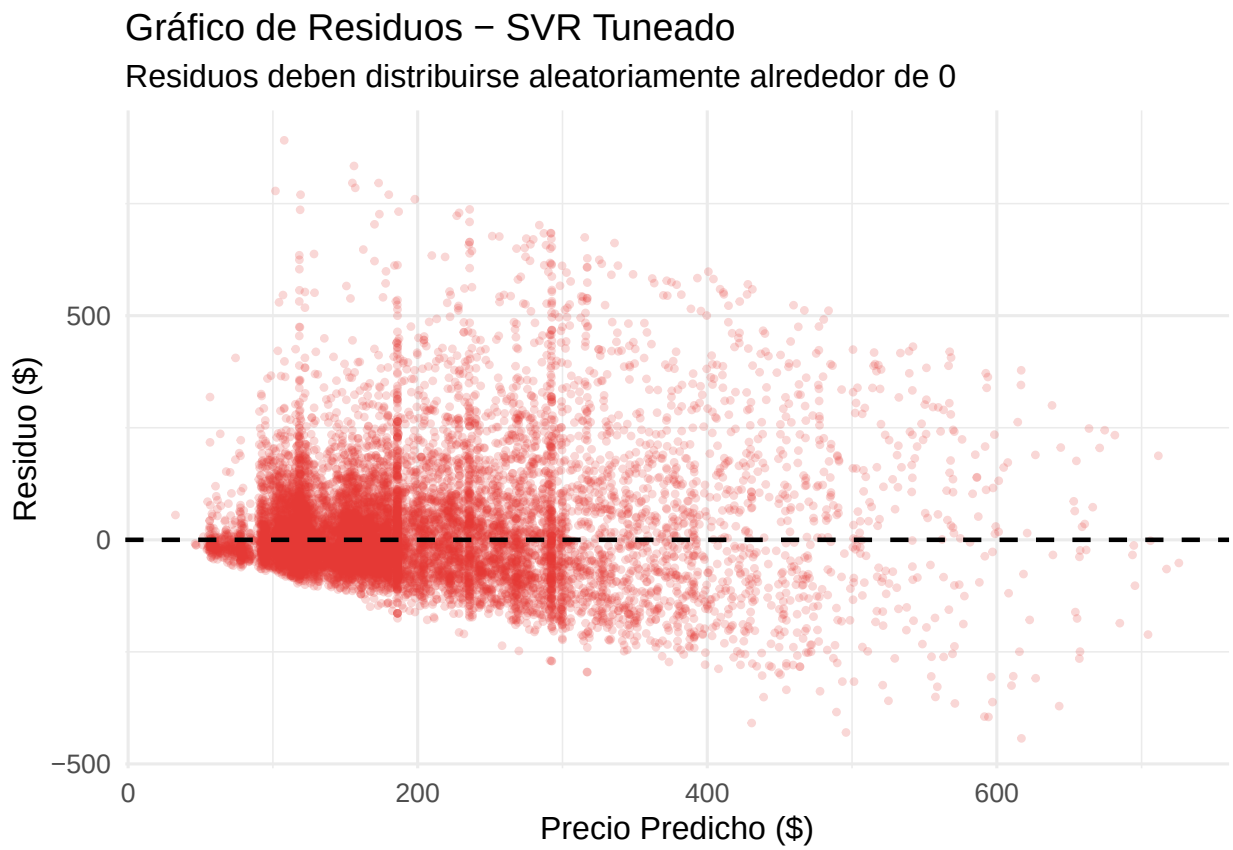
```
## SVR Tuneado               135.12      87.69     0.4029
```

```r
# Gráfico Real vs Predicho
df_pred <- data.frame(
  Real     = test_modelo$price,
  Predicho = pred_tuned_reg
)

ggplot(df_pred, aes(x = Real, y = Predicho)) +
  geom_point(alpha = 0.2, color = "#1565C0", size = 0.8) +
  geom_abline(slope = 1, intercept = 0,
              color = "red", linewidth = 1, linetype = "dashed") +
  labs(title    = "SVR Tuneado - Precio Real vs Predicho",
       subtitle = "La línea roja representa predicción perfecta",
       x = "Precio Real ($)", y = "Precio Predicho ($)") +
  theme_minimal(base_size = 12)
```




```r
# Gráfico de residuos
df_pred$Residuo <- df_pred$Real - df_pred$Predicho

ggplot(df_pred, aes(x = Predicho, y = Residuo)) +
```

```r
  geom_point(alpha = 0.2, color = "#E53935", size = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed",
             color = "black", linewidth = 0.8) +
  labs(title    = "Gráfico de Residuos - SVR Tuneado",
       subtitle = "Residuos deben distribuirse aleatoriamente alrededor de 0",
       x = "Precio Predicho ($)", y = "Residuo ($)") +
  theme_minimal(base_size = 12)
```



El modelo SVR utilizó 4,118 vectores de soporte sobre las 5,000 observaciones de entrenamiento, obteniendo un RMSE de \$135.23, MAE de \$87.59 y R^2 de 0.4023, lo que significa que solo explica el 40% de la varianza del precio con un error promedio de casi \$88 por propiedad. El tuneo mediante grid search identificó sigma=0.05 y C=1 como la mejor combinación, pero la mejora fue prácticamente nula ya que el RMSE bajó apenas \$0.11, el MAE aumentó marginalmente y el R^2 mejoró en solo 0.0006, confirmando que el modelo ya estaba cerca de su límite de capacidad predictiva con las variables disponibles y que ajustar hiperparámetros no puede compensar la ausencia de predictores más informativos como ubicación geográfica, tipo de propiedad o características del anfitrión.

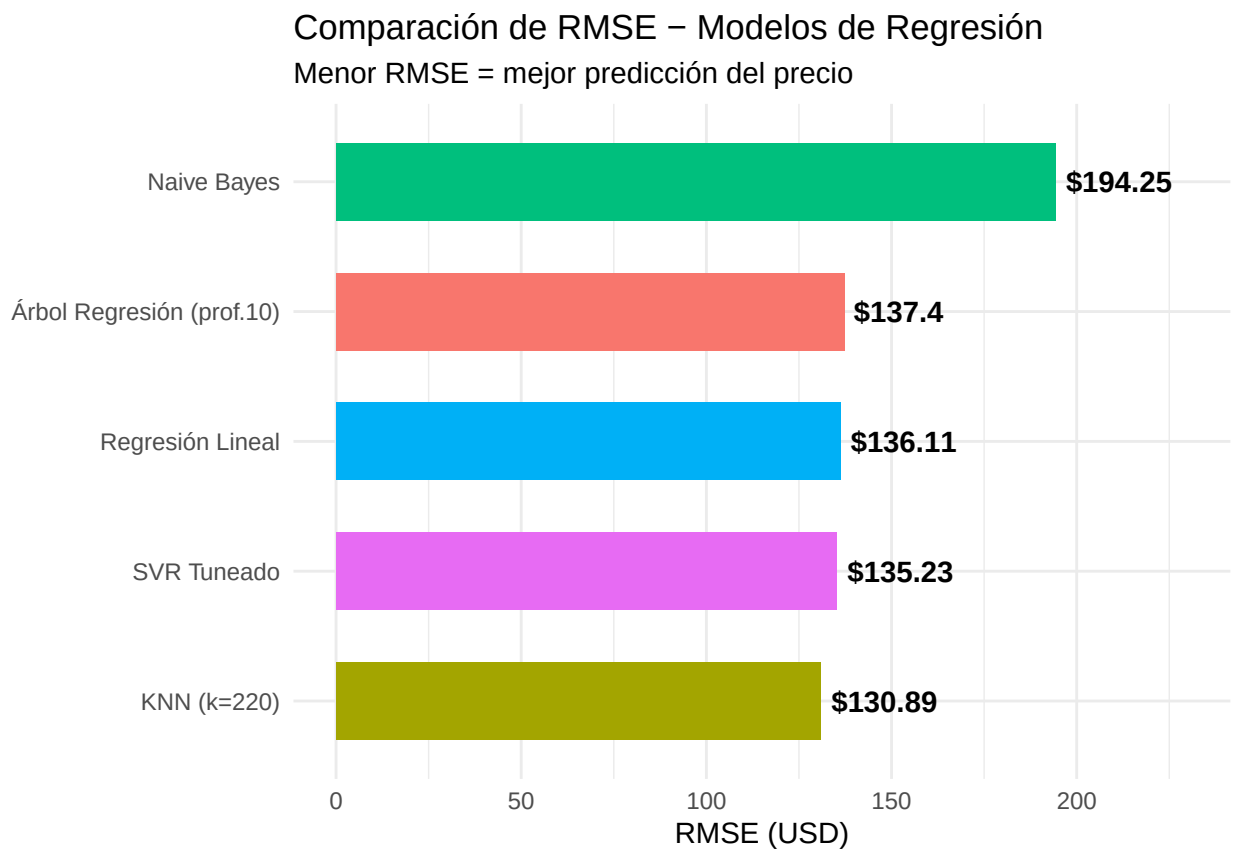
Actividad 12. Comparacion de Resultados

```r
# Actividad 12: Comparación SVR con modelos de regresión anteriores

comparacion_reg_final <- data.frame(
  Modelo = c("Naive Bayes",
```

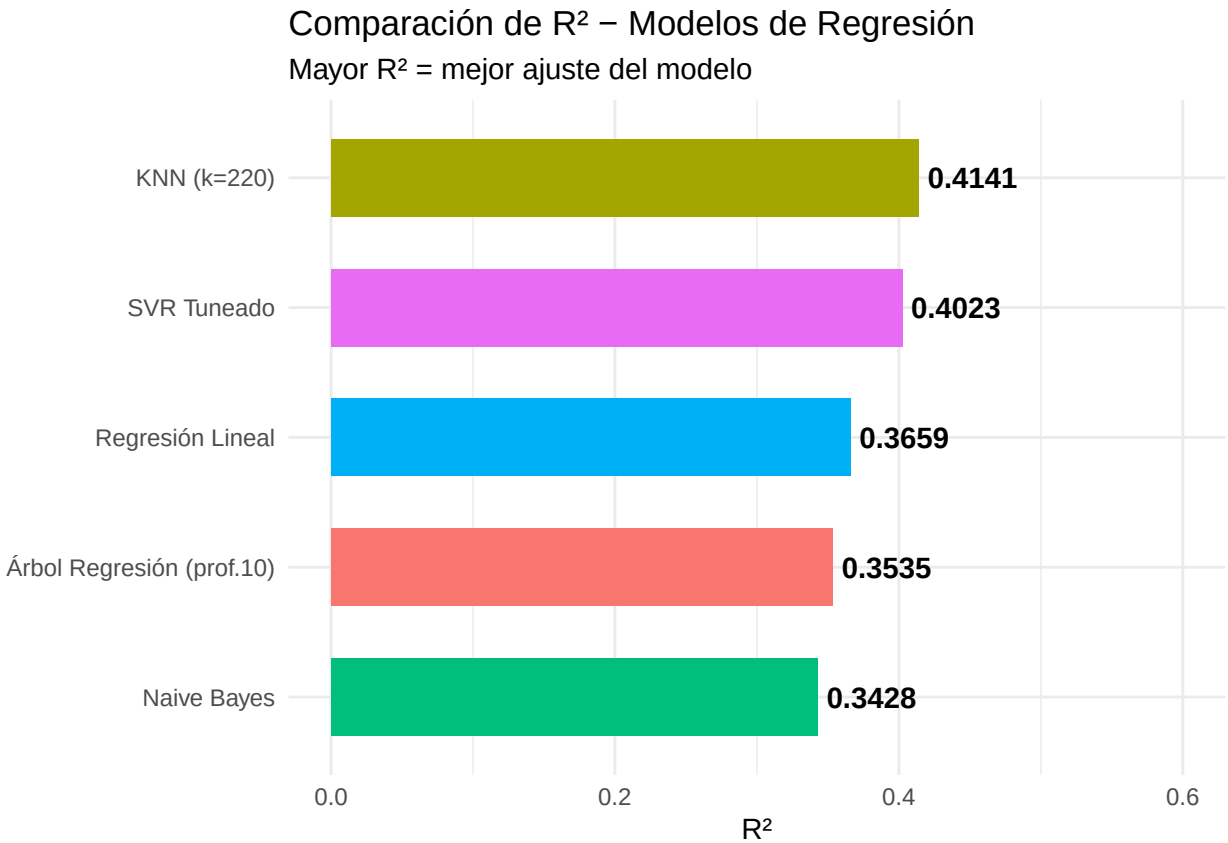
```r
             "Árbol Regresión (prof.10)",
             "Regresión Lineal",
             "SVR Tuneado",
             "KNN (k=220)"),
  RMSE   = c(194.25, 137.40, 136.11, 135.23, 130.89),
  MAE    = c(132.59,  95.50,  94.95,  87.59,  90.81),
  R2     = c(0.3428,  0.3535, 0.3659, 0.4023, 0.4141)
)

print(comparacion_reg_final)
```

```
##                       Modelo   RMSE    MAE     R2
## 1                Naive Bayes 194.25 132.59 0.3428
## 2 Árbol Regresión (prof.10) 137.40  95.50 0.3535
## 3          Regresión Lineal 136.11  94.95 0.3659
## 4                SVR Tuneado 135.23  87.59 0.4023
## 5                KNN (k=220) 130.89  90.81 0.4141
```

```r
# Gráfico RMSE
ggplot(comparacion_reg_final,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0("$", RMSE)),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 230) +
  labs(title    = "Comparación de RMSE - Modelos de Regresión",
       subtitle = "Menor RMSE = mejor predicción del precio",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```




```r
# Gráfico R²
ggplot(comparacion_reg_final,
       aes(x = reorder(Modelo, R2), y = R2, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = R2), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 0.6) +
  labs(title    = "Comparación de R² - Modelos de Regresión",
       subtitle = "Mayor R² = mejor ajuste del modelo",
       x = NULL, y = "R²") +
  theme_minimal()
```



La comparación final entre todos los modelos de regresión evaluados a lo largo del curso muestra que KNN sigue siendo el mejor modelo con el menor RMSE de \$130.89 y el mayor R^2 de 0.4141, superando a todos los demás algoritmos. El SVR Tuneado queda en segundo lugar con RMSE de \$135.23 y R^2 de 0.4023, un resultado competitivo que lo coloca por encima de la Regresión Lineal (\$136.11, R^2 0.3659) y del Árbol de Regresión (\$137.40, R^2 0.3535). Es destacable que el SVR logra el mejor MAE de todos los modelos con \$87.59, superando incluso a KNN (\$90.81), lo que significa que en términos de error absoluto promedio el SVR comete errores más pequeños aunque su RMSE sea ligeramente mayor.

Naive Bayes sigue siendo claramente el peor modelo de regresión con RMSE de \$194.25 y R^2 de apenas 0.3428, confirmando que no es un algoritmo adecuado para predicción de valores continuos. En general, los cuatro modelos competitivos (KNN, SVR, Regresión Lineal y Árbol) se encuentran en un rango estrecho de RMSE entre \$130 y \$138, lo que sugiere que la limitación real no está en el algoritmo sino en las variables disponibles. Ningún modelo logra superar el 42% de varianza explicada con las 5 variables numéricas actuales, por lo que para mejorar significativamente los resultados sería necesario incorporar variables como ubicación geográfica, tipo de propiedad o características del anfitrión que capturen mejor los determinantes reales del precio en Airbnb.

Actividad 13. Conclusiones

A lo largo de este laboratorio se evaluaron modelos de SVM para clasificación y SVR para regresión aplicados al dataset de propiedades Airbnb de SmartStay Advisors. En clasificación, los cuatro modelos SVM probados (Lineal, Radial, Polinomial y Tuneado) obtuvieron resultados modestos en el rango de 52-58% de Accuracy, todos diagnosticados con desajuste debido a que las 5 variables numéricas disponibles no son suficientes para separar claramente las tres categorías de precio. El mejor modelo SVM fue el Tuneado con kernel radial, C=10 y sigma=0.01 con 58.41% de Accuracy y Kappa 0.3772, aunque no logró superar al Árbol de Decisión

(69.24%), la Regresión Logística Binaria (76.30%) ni a Random Forest (84.46%), que sigue siendo el mejor clasificador de todos los labs.

En regresión, el SVR Tuneado obtuvo un RMSE de $135.23 y R^2 de 0.4023, quedando en segundo lugar detrás de KNN ($130.89, R^2 0.4141) pero superando a la Regresión Lineal, el Árbol de Regresión y Naive Bayes. Destaca que el SVR logró el mejor MAE de todos los modelos con $87.59, lo que lo convierte en una alternativa competitiva cuando se prioriza minimizar el error absoluto promedio sobre el error cuadrático.

Para los objetivos de SmartStay Advisors se recomienda Random Forest para clasificación por su clara superioridad, y KNN para regresión por su menor RMSE, aunque el SVR es una alternativa válida especialmente cuando el tiempo de entrenamiento es una restricción, ya que con muestras reducidas es considerablemente más rápido que KNN con datasets completos.